

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Báo Cáo Đồ Án 2: Xây Dựng Hệ Thống CD

Đề tài: Xây dựng hệ thống CD, GitOps, Service Mesh và Observability
cho YAS

Môn học: DevOps

Sinh viên thực hiện:

Lê Xuân Trí (23120099)

Phạm Quang Vinh (23120202)

Nguyễn Lê Thế Vinh (23120190)

Ngày 7 tháng 7 năm 2026

Mục lục

1	Bảng Thuật Ngữ Tiếng Anh - Tiếng Việt	1
2	Thông Tin Nhóm	2
3	Phân Công Công Việc	2
4	Checklist Yêu Cầu Đồ Án 2	2
4.1	Hai Repository Trong Phạm Vi Báo Cáo	3
4.2	Traceability Từ Yêu Cầu Đề Bài Đến Evidence	4
5	Workflow Architecture Của Hệ Thống CD	6
5.1	Sơ Đồ Source-To-Runtime	6
5.2	Ranh Giới Trách Nhiệm Giữa Hai Repo	7
5.3	Luồng Quyết Định Theo Trigger	7
5.4	Rollback Và Audit Trail	7
6	Kiến Trúc Tổng Quan	8
6.1	Luồng End-to-End	8
6.2	Hạ Tầng Triển Khai	9
6.3	Runtime Policy Trên Single-Node	9
7	Phân Tích Thay Đổi Trong App Repo tzin1401/yas	11
7.1	Vai Trò Của App Repo Trong Kiến Trúc Split-Repo	11
7.2	Jenkinsfile Chính: CI Gate Và CD Trigger	12
7.3	Chiến Lược Tag Image	12
7.4	Release Promote Từ Dev Overlay	13
7.5	Service Catalog Trong App Repo	13
7.6	Changed-Module Detection	13
7.7	Build Và Runtime Fixes	14
7.8	Observability Instrumentation Trong App Repo	14
8	CD Repo Và GitOps Desired State	15

8.1	Cấu Trúc Desired State	15
8.2	CQ Service Scope	15
8.3	Validation Gates Trong CD Repo	16
8.4	ArgoCD Sync Policy	17
8.5	Active Environment Policy	17
9	Các Luồng Jenkins CD	18
9.1	Dev Auto Deploy	18
9.2	Staging Release	19
9.3	Developer Preview	21
9.4	Teardown Developer	23
9.5	Bảng Chứng Demo 3 Luồng	23
10	Runtime Verification Và Service Mesh	23
10.1	Platform Readiness	23
10.2	External Access	24
10.3	Service Mesh Scope	25
10.4	AuthorizationPolicy Evidence	26
10.5	Kiali Topology	28
11	Observability Với Prometheus Và Grafana	28
11.1	Mục Tiêu Observability	28
11.2	Instrumentation Trong App Repo	29
11.3	Tự Động Tiêm Agent OpenTelemetry (Auto-Instrumentation)	29
11.4	Scrape Contract Trong CD Repo	30
11.5	Local Observability Assets Từ App Repo	30
11.6	Runtime Evidence Cho Observability	31
11.7	Evidence Cần Bổ Sung	31
12	Evidence Tổng Hợp Và Known Issues	33
12.1	Evidence Đã Có	33
12.2	Ảnh Evidence Đã Đưa Vào Report	34
12.3	Ảnh Cần Chụp Bổ Sung Theo Ưu Tiên	35

12.4 Các Sự Cố Kỹ Thuật Và Giải Pháp Khắc Phục (Troubleshooting)	35
12.5 Known Issues Và Cách Trình Bày Trung Thực	37
12.6 Kết Luận	37
Tài liệu tham khảo	38

Danh sách hình vẽ

1	Workflow source-to-runtime của hệ thống CD	6
2	GCP VM 32 GB RAM dùng làm Jenkins agent và K3s node	10
3	ArgoCD quản lý các app YAS trong CD repo	10
4	Scaffold ảnh app repo Jenkins pipeline overview	11
5	Staging release tạo diff trong ArgoCD trước khi operator sync thủ công	17
6	Scaffold ảnh detail ArgoCD cho <code>yas-dev</code> , <code>yas-staging</code> , <code>yas-developer</code>	18
7	Scaffold ảnh Jenkins main build chạy qua CI gates và Docker push	19
8	Scaffold ảnh Docker Hub hiển thị image tag theo commit SHA cho dev	19
9	Scaffold ảnh Jenkins release tag <code>v0.1.2</code> promote thành công	20
10	ArgoCD staging có diff trước khi operator sync thủ công	20
11	Developer preview truy cập được qua NodePort 30846	22
12	Scaffold ảnh job <code>developer_build</code> với parameter <code>branch/service</code>	22
13	Scaffold ảnh ArgoCD/Kubernetes khi developer active và staging dormant	22
14	Scaffold ảnh job teardown trả developer về dormant	23
15	External IP của GCP VM dùng cho NodePort demo	24
16	Storefront dev reachable qua NodePort	25
17	Scaffold ảnh pod có workload container và Istio sidecar <code>READY 2/2</code>	26
18	Scaffold ảnh curl allow: <code>tax</code> gọi <code>location</code>	27
19	Scaffold ảnh curl deny: <code>cart</code> gọi <code>location</code> bị Envoy RBAC 403	27
20	Scaffold ảnh retry policy hoặc log retry cho service trong critical path	27
21	Placeholder ảnh Kiali topology cần bổ sung trước khi nộp	28
22	Luồng observability từ service đến dashboard	29
23	Placeholder ảnh Prometheus targets cần bổ sung	32
24	Scaffold ảnh Grafana datasource Prometheus healthy	32
25	Placeholder ảnh Grafana dashboard cần bổ sung	32
26	Scaffold ảnh OpenTelemetry Collector/Operator chạy trên cluster	33

Danh sách bảng

1	Bảng tra cứu thuật ngữ chuyên ngành	1
2	Thông tin thành viên nhóm	2
3	Phân công công việc các thành viên	2
4	Đối chiếu yêu cầu bắt buộc của Đồ án 2	3
5	Yêu cầu nâng cao và phần bổ sung	3
6	Ma trận yêu cầu, triển khai và evidence	5
7	Boundary giữa app repo, CD repo và runtime	7
8	Trigger, tag image và chính sách sync	7
9	Các thành phần hạ tầng chính	9
10	Contract app repo xuất sang CD flow	11
11	Chiến lược image tag trong app repo	12
12	Vai trò các thư mục GitOps chính	15
13	Service scope runtime theo CQ demo	16
14	Quá trình hardening staging release	21
15	Service mesh requirement và implementation	26
16	Checklist runtime observability evidence	31
17	Nhóm evidence hiện có	33
18	Ảnh evidence thật đang hiển thị trong PDF	34
19	Backlog ảnh cần bổ sung trước khi nộp	35

1 Bảng Thuật Ngữ Tiếng Anh - Tiếng Việt

Bảng 1: Bảng tra cứu thuật ngữ chuyên ngành

Thuật ngữ	Dịch nghĩa	Giải thích / Ý nghĩa
GitOps	Vận hành qua Git	Phương pháp quản lý hạ tầng và ứng dụng thông qua Git làm nguồn sự thật duy nhất (source of truth).
Continuous Delivery (CD)	Chuyển giao liên tục	Quá trình tự động hóa việc đưa mã nguồn đã qua kiểm thử lên các môi trường runtime.
Service Mesh	Lưới dịch vụ	Lớp hạ tầng chuyên dụng để quản lý giao tiếp giữa các dịch vụ (microservices), đảm bảo bảo mật và khả năng giám sát.
mTLS (Mutual TLS)	TLS hai chiều	Giao thức xác thực bảo mật hai chiều giữa client và server thông qua chứng chỉ mã hóa.
Ingress Controller	Bộ điều khiển Ingress	Thành phần quản lý và định tuyến lưu lượng truy cập từ ngoài vào trong Kubernetes cluster.
Auto-sync	Tự động đồng bộ	Cơ chế tự động đối chiếu và cập nhật trạng thái thực tế khớp với Git mong muốn của ArgoCD.
BFF (Backend for Frontend)	Backend chuyên biệt cho Frontend	Dịch vụ trung gian tối ưu hóa dữ liệu và định tuyến giữa client và các microservices.
Instrumentation	Đo lường / Cài cắm mã	Việc tích hợp mã nguồn hoặc tác nhân (agent) để thu thập số liệu giám sát (metrics, traces).
Quality Gate	Cổng kiểm soát chất lượng	Ngưỡng tiêu chí (ví dụ: SonarQube, JaCoCo) mà mã nguồn bắt buộc phải vượt qua để được đi tiếp.
Pipeline	Đường ống tự động	Chuỗi các bước tự động hóa từ biên dịch, kiểm thử, phân tích cho đến đóng gói.

2 Thông Tin Nhóm

Bảng 2: Thông tin thành viên nhóm

Thành viên	MSSV	Phần phụ trách
Lê Xuân Trí	23120099	Cluster, CD integration, K3s, ArgoCD và mesh evidence
Phạm Quang Vinh	23120202	Jenkins, Docker image pipeline, credential binding và GitOps update
Nguyễn Lê Thế Vinh	23120190	GitOps manifests, security/RBAC audit, report và evidence

App/CI Repository: <https://github.com/tzin1401/yas>

CD/GitOps Repository: <https://github.com/emanhthangngot/yas-cd>

3 Phân Công Công Việc

Bảng 3: Phân công công việc các thành viên

Thành viên	Nội dung thực hiện
Lê Xuân Trí	Phụ trách hạ tầng GCP VM, K3s single-node, bootstrap ArgoCD từ CD repo, kiểm tra trạng thái cluster, thu thập evidence runtime và service mesh. Hoàn thành khi VM/cluster chạy ổn định, ArgoCD apps Healthy/Synced hoặc có giải thích lệch trạng thái, mesh evidence được ghi nhận.
Phạm Quang Vinh	Phụ trách Jenkins và image pipeline trong app repo tzin1401/yas : Jenkinsfile, Docker Hub build/push, binding credentials, chiến lược image tag và job cập nhật overlay trong CD repo. Hoàn thành khi CI/CD jobs chạy được, image tags đúng, deploy/rollback/smoke job logs được lưu làm evidence.
Nguyễn Lê Thế Vinh	Phụ trách GitOps, security và report trong CD repo emanhthangngot/yas-cd : base/overlays, ArgoCD applications, secret/RBAC audit, validate GitOps, tổng hợp báo cáo và evidence. Hoàn thành khi CD repo render được, YAML đúng service scope, staging dùng immutable tags, report hoàn chỉnh.

4 Checklist Yêu Cầu Đồ Án 2

Báo cáo này bám theo đề bài Đồ án 2: xây dựng hệ thống CD cho YAS, có Jenkins, Docker Hub, Kubernetes, ArgoCD, Service Mesh và phần Observability bổ sung. Phần CI của Đồ án 1 không bị

thay thế; nó được giữ lại như lớp kiểm soát chất lượng trước khi image được đưa vào CD.

Bảng 4: Đối chiếu yêu cầu bắt buộc của Đồ án 2

#	Yêu cầu	Trạng thái
1	Kubernetes cluster cho YAS trên GCP VM chạy K3s, dùng NodePort để demo ứng dụng.	Đã có evidence
2	Mỗi branch/service thay đổi tạo Docker image tag theo commit SHA và push lên Docker Hub.	Đã triển khai trong app repo
3	Main branch tự deploy vào môi trường dev.	Đã triển khai qua Jenkins + GitOps + ArgoCD
4	Release tag vX.Y.Z deploy vào <code>staging</code> .	Đã chạy thành công với v0.1.2
5	Job <code>developer_build</code> cho phép chọn branch từng service để preview.	Đã triển khai trong CD repo
6	Job teardown để xóa developer preview.	Đã triển khai trong CD repo
7	Không deploy trực tiếp bằng <code>kubectl</code> ; Jenkins chỉ cập nhật GitOps repo, ArgoCD sync cluster.	Đã áp dụng

Bảng 5: Yêu cầu nâng cao và phần bổ sung

#	Nội dung	Trạng thái
A1	ArgoCD quản lý <code>dev</code> , <code>staging</code> , <code>developer</code> , platform và <code>sampledata</code> seed app.	Đã có evidence
A2	Service Mesh với Istio: mTLS, retry policy, AuthorizationPolicy allow/deny, Kiali topology.	Đã có evidence chính; Kiali screenshot cần bổ sung nếu thiếu
A3	Observability Prometheus/Grafana: app expose metrics, CD chart có ServiceMonitor contract.	Instrumentation đã có; runtime screenshot cần bổ sung
A4	Bảo toàn CI gates Đồ án 1: Gitleaks, test, coverage, SonarQube; Snyk ghi nhận caveat hiện tại.	Đã giữ, Snyk tạm tắt

4.1 Hai Repository Trong Phạm Vi Báo Cáo

- App/CI repo: <https://github.com/tzin1401/yas.git>. Repository này chứa source code YAS, Jenkinsfile chính, `services.yaml`, Dockerfile, CI scripts, test, và phần instrumentation

cho Prometheus/OpenTelemetry.

- CD/GitOps repo: <https://github.com/emanhthangngot/yas-cd.git>. Repository này chứa desired state cho Kubernetes: `base/`, `overlays/`, `argocd/`, `charts/`, Jenkins job GitOps nhẹ và evidence.

Ranh giới quan trọng: app repo sinh ra image và trigger GitOps commit; CD repo là nguồn sự thật cho trạng thái cluster. Nhóm không dùng `kubectl set image` hay `kubectl apply` trực tiếp vào namespace do ArgoCD quản lý.

4.2 Traceability Từ Yêu Cầu Đề Bài Đến Evidence

Bảng dưới đây gom các yêu cầu quan trọng nhất của đề bài thành các mục có thể kiểm chứng. Khi bảo vệ, nhóm nên mở theo thứ tự này để người chấm thấy rõ mỗi yêu cầu được đáp ứng bởi phần nào trong hệ thống.

Bảng 6: Ma trận yêu cầu, triển khai và evidence

Yêu cầu	Triển khai	Evidence/đường dẫn
Kubernetes cluster	K3s single-node trên GCP VM 32GB; workload và Jenkins agent cùng chạy trên VM này.	img/platform/01_gcp_vm_specs.png, img/platform/02_gcp_vm_ip_addr.png.
Branch image tag	Feature branch build image theo short commit SHA; Docker Hub là registry cuối.	App repo Jenkinsfile, services.yaml, Docker Hub tag screenshot cần bổ sung.
Dev auto deploy	main build thành công sẽ update overlays/dev bằng commit SHA.	GitOps commit update dev image tags, ArgoCD yas-dev screenshot.
Staging release	Git tag vX.Y.Z promote image từ dev overlay sang immutable release tag.	21d_jenkins_v012_release_success.txt, img/argocd/24_argocd_outofsync_diff.png.
Developer preview	CD repo có Jenkinsfile.developer-build.yml, service branch được chọn bằng parameter.	28_developer_build_console.txt, 29_preview_commit.txt, img/developer/32_developer_reachable.png.
Teardown preview	Job teardown reset developer overlay về dormant/default, không dùng kubectl delete.	33_teardown_console.txt.
ArgoCD nâng cao	ArgoCD quản lý dev, staging, developer, platform và sampledata seed app.	img/argocd/06_argocd_apps_overview.png, 07_validate_gitops_passed.txt.
Service mesh	Istio mTLS, DestinationRule, VirtualService retry và AuthorizationPolicy allow/deny.	overlays/dev/istio, overlays/staging/istio, Kiali/curl screenshots cần bổ sung.
Observability	App expose Actuator/Prometheus, CD chart có ServiceMonitor, OpenTelemetry auto-instrumentation.	Prometheus/Grafana screenshots cần bổ sung; app repo docker-compose.011y.yml.

5 Workflow Architecture Của Hệ Thống CD

Phần này mô tả logic vận hành của đồ án ở mức workflow. Mục tiêu chính không chỉ là “deploy được”, mà là mọi thay đổi runtime đều có provenance: commit nguồn trong app repo, image tag trên Docker Hub, commit GitOps trong CD repo, và trạng thái reconcile trong ArgoCD.

5.1 Sơ Đồ Source-To-Runtime



Hình 1: Workflow source-to-runtime của hệ thống CD

5.2 Ranh Giới Trách Nhiệm Giữa Hai Repo

Bảng 7: Boundary giữa app repo, CD repo và runtime

Lớp	Chịu trách nhiệm	Không chịu trách nhiệm
App/CI repo	Source code YAS, Jenkins-file chính, CI gate, Docker build/push, <code>services.yaml</code> , app instrumentation.	Không lưu desired state cuối cùng của cluster; không apply trực tiếp vào namespace ArgoCD quản lý.
CD/GitOps repo	Kustomize base/overlays, chart snapshot, ArgoCD apps, Jenkins job GitOps nhẹ, sampledata seed job, evidence/runbook/report.	Không build source code Java/Next.js chính; không thay thế CI gate của app repo.
Runtime K3s	Chạy workload, platform services, ingress, mesh, observability, ArgoCD reconcile.	Không là nơi chỉnh image bằng tay; trạng thái mong muốn phải quay về Git.

5.3 Luồng Quyết Định Theo Trigger

Bảng 8: Trigger, tag image và chính sách sync

Trigger	Image tag	GitOps action	ArgoCD policy
Feature branch	Short commit SHA 12 ký tự.	Không update dev/staging; developer preview dùng job riêng để chọn service/branch.	<code>yas-developer</code> auto-sync khi preview active.
main	Commit SHA, <code>main</code> , <code>latest</code> .	Update <code>overlays/dev</code> bằng commit SHA để tạo manifest diff thật.	<code>yas-dev</code> auto-sync/self-heal.
vX.Y.Z	Promote image hiện có sang release tag.	Update <code>overlays/staging</code> bằng immutable tag.	<code>yas-staging</code> manual-sync để operator duyệt.
Teardown developer	Không build image mới.	Reset <code>overlays/developer</code> về dormant/default tags.	<code>yas-developer</code> reconcile về 0/0.

5.4 Rollback Và Audit Trail

Rollback trong mô hình này không dùng `kubectl set image`. Nhóm có thể rollback bằng cách revert GitOps commit hoặc chỉnh lại tag trong overlay rồi để ArgoCD reconcile. Chuỗi audit cần đọc theo thứ tự:

1. App commit hoặc release tag trong `tzin1401/yas`.
2. Jenkins console log chứng minh CI gate/build/promote.
3. Docker Hub tag tồn tại cho service tương ứng.
4. GitOps commit trong `emanhthangngot/yas-cd`.
5. ArgoCD sync status và Kubernetes rollout/runtime smoke.

Đây là điểm khác biệt quan trọng so với cách deploy thủ công: trạng thái runtime có thể giải thích ngược về Git và image tag, không phụ thuộc thao tác trên cluster.

6 Kiến Trúc Tổng Quan

Hệ thống được thiết kế theo mô hình split-repo: app repo xử lý source code, quality gates và Docker image; CD repo lưu desired state để ArgoCD đồng bộ vào Kubernetes. Cách tách này giúp Jenkins không trực tiếp thay đổi workload trên cluster, đồng thời mọi thay đổi deploy đều có lịch sử Git rõ ràng.

6.1 Luồng End-to-End

```

1 Developer push branch/tag -> Jenkins app repo pipeline
2   -> Gitleaks/Test/Coverage/SonarQube gate
3   -> Docker Hub image docker.io/<user>/yas-<service>:<tag>
4   -> Jenkins commits overlay change to yas-cd/main
5   -> ArgoCD detects Git change
6   -> K3s reconciles dev/staging/developer namespaces

```

Listing 1: Luồng source-to-runtime của đồ án

6.2 Hạ Tầng Triển Khai

Bảng 9: Các thành phần hạ tầng chính

Thành phần	Vai trò
Jenkins Controller	Chạy trên AWS EC2, điều phối pipeline và lưu log build.
Jenkins Agents	App repo CI nặng chạy trên label <code>yas-build-worker</code> ; GitOps jobs nhẹ trong CD repo chạy trên <code>gcp-build-agent</code> .
GCP VM	Máy <code>gcp-ci-cd-agent</code> chạy K3s single-node, Jenkins agent, ArgoCD và workload demo.
K3s	Kubernetes runtime cho <code>dev</code> , <code>staging</code> , <code>developer</code> , platform services và mesh.
Docker Hub	Registry cuối cùng của image: <code>docker.io/<DOCKERHUB_USERNAME>/yas-<service>:<tag></code> .
ArgoCD	Đọc <code>yas-cd/main</code> và reconcile desired state vào cluster.
Istio/Kiali	Service mesh cho mTLS, retry policy, AuthorizationPolicy và topology.
Prometheus/Grafana	Observability bổ sung: app expose metric, Prometheus scrape, Grafana dashboard.

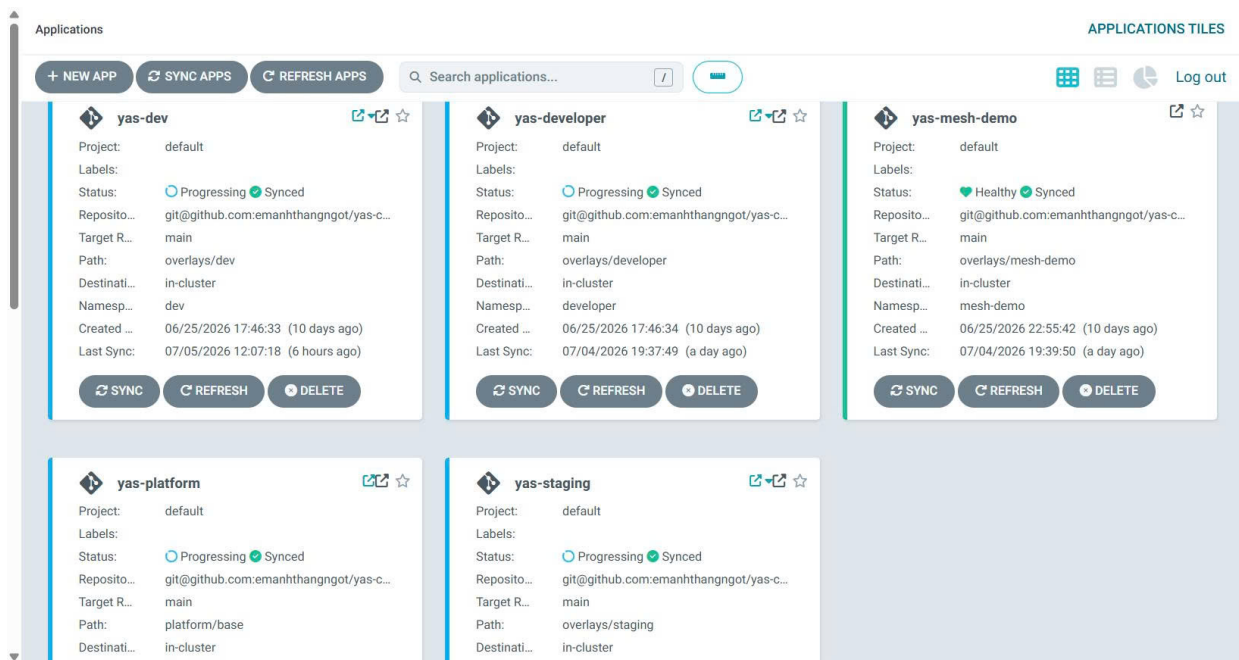
6.3 Runtime Policy Trên Single-Node

Do toàn bộ hệ thống chạy trên một VM duy nhất, nhóm không giữ ba môi trường full-stack cùng active. Baseline là:

- `dev`: active, auto-sync.
- `staging`: active nhưng manual sync cho release approval.
- `developer`: dormant, chỉ bật khi chạy developer preview.
- Khi `developer` active, `staging` chuyển dormant để tránh quá tải single-node.

```
xuantri@gcp-ci-cd-agent:~$ hostname
gcp-ci-cd-agent
xuantri@gcp-ci-cd-agent:~$ lsb-release -a || cat /etc/os-release
-bash: lsb-release: command not found
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
xuantri@gcp-ci-cd-agent:~$ nproc
8
xuantri@gcp-ci-cd-agent:~$ free -h
              total        used        free     shared  buff/cache   available
Mem:           31Gi        2.1Gi        23Gi        3.4Mi        5.8Gi        29Gi
Swap:            0B           0B           0B
xuantri@gcp-ci-cd-agent:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        96G   6.9G   89G   8% /
tmpfs           16G     0    16G   0% /dev/shm
tmpfs           6.3G   3.3M   6.3G   1% /run
tmpfs           5.0M     0   5.0M   0% /run/lock
efivarfs        256K   32K   220K  13% /sys/firmware/efi/efivars
/dev/sda16      881M   51M   769M   7% /boot
/dev/sda15      105M   6.2M   99M    6% /boot/efi
tmpfs           3.2G   8.0K   3.2G   1% /run/user/1001
tmpfs           3.2G   8.0K   3.2G   1% /run/user/1002
```

Hình 2: GCP VM 32 GB RAM dùng làm Jenkins agent và K3s node



Hình 3: ArgoCD quản lý các app YAS trong CD repo

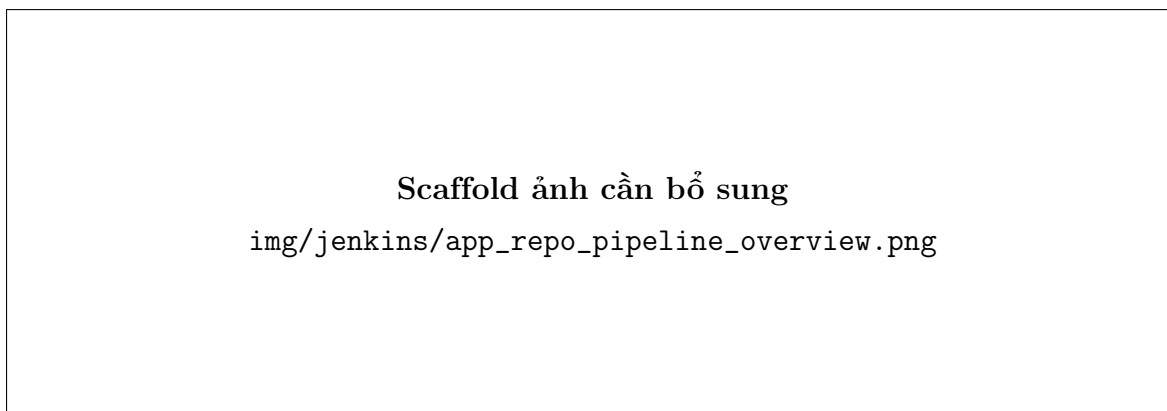
7 Phân Tích Thay Đổi Trong App Repo tzin1401/yas

App repo không chỉ là source code gốc của YAS. Trong đồ án này, app repo đã được thay đổi để trở thành điểm xuất phát của CD: phát hiện module thay đổi, chạy CI gates, build Docker image, push Docker Hub, rồi cập nhật CD repo bằng GitOps. Các nội dung dưới đây cần được đưa vào báo cáo vì chúng giải thích vì sao CD repo có thể deploy đúng image và đúng service.

7.1 Vai Trò Của App Repo Trong Kiến Trúc Split-Repo

App repo là nơi tạo artifact deployable, nhưng không phải nơi lưu trạng thái mong muốn cuối cùng của cluster. Contract giữa app repo và CD repo gồm ba dữ liệu:

Bảng 10: Contract app repo xuất sang CD flow		
Dữ liệu	Nguồn trong app repo	Cách CD repo sử dụng
Service catalog	<code>services.yaml</code>	Jenkins biết service nào deployable, image name, Dockerfile và dependency expansion.
Image tag	Branch/main/tag context trong Jenkinsfile	CD repo overlay nhận tag commit SHA hoặc release tag immutable.
CI verdict	Gitleaks, test, coverage, SonarQube, build result	Chỉ image đã qua quality gate mới được dùng làm nguồn deploy/promote.



Hình 4: Scaffold ảnh app repo Jenkins pipeline overview

7.2 Jenkinsfile Chính: CI Gate Và CD Trigger

Jenkinsfile trên app repo `origin/main` dùng agent label `yas-build-worker` cho các bước nặng như test, build, scan và Docker. Pipeline chính có các nhóm stage:

- Checkout, fetch `origin/main`, chmod Maven wrapper và kiểm tra tool.
- Detect changed modules bằng `ci/detect-changed-modules.sh`.
- Gitleaks, test, JaCoCo report, coverage gate và SonarQube.
- Docker Hub build/push image cho các service deployable.
- GitOps update: clone `emanhthangngot/yas-cd`, chỉnh overlay, commit và push vào `main`.

Một điểm phải ghi rõ: stage Snyk hiện đang bị tắt tạm thời bằng điều kiện `when { expression { false } }` để unblock CD validation. Vì vậy báo cáo không claim Snyk đang chạy ở bản app repo hiện tại; chỉ ghi đây là CI gate kế thừa từ Đồ án 1 và cần bật lại sau khi CD ổn định.

7.3 Chiến Lược Tag Image

Bảng 11: Chiến lược image tag trong app repo

Nguồn trigger	Image tag được push	GitOps target
Feature branch	Short commit SHA 12 ký tự	Không update GitOps trực tiếp; developer preview dùng job riêng
<code>main</code>	Short commit SHA, <code>main</code> , <code>latest</code>	Update <code>overlays/dev</code> bằng short commit SHA
<code>vX.Y.Z</code> tag	Retag image hiện có sang <code>vX.Y.Z</code>	Update <code>overlays/staging</code> bằng immutable release tag

Việc dev dùng commit SHA thay vì `main/latest` là thay đổi quan trọng: mỗi lần main build thành công đều tạo Deployment template diff thật, ArgoCD có lý do rollout pod mới, đồng thời rollback có thể truy ngược về tag bất biến.

7.4 Release Promote Từ Dev Overlay

Trong giai đoạn đầu, release tag cố promote mọi service từ cùng một tagged commit SHA. Cách này lỗi khi dev đang là trạng thái incremental: một số service đang ở SHA mới, một số service vẫn ở tag cũ. App repo đã sửa Jenkinsfile để đọc tag nguồn từng service từ `overlays/dev/kustomization.yaml` trong CD repo, sau đó dùng `docker buildx imagetools create` retag từng image sang release `vX.Y.Z`. Nhờ đó release `v0.1.2` promote thành công 15 service.

7.5 Service Catalog Trong App Repo

File `services.yaml` trong app repo là contract cho Jenkins:

- Xác định service nào deployable, Dockerfile nằm ở đâu, imageName là gì.
- Ghi demo role: teacher-required, runtime-dependency, optional, run-once.
- Ghi dependencies để CI có thể mở rộng phạm vi build/test khi dependency thay đổi.

Các service deployable chính là `cart`, `customer`, `inventory`, `location`, `media`, `order`, `payment`, `payment-paypal`, `product`, `search`, `tax`, `backoffice-bff`, `storefront-bff`, `backoffice-ui`, `storefront-ui`. Các service `promotion`, `rating`, `recommendation`, `webhook` được giữ trong source nhưng không thuộc runtime CQ mặc định. `sampledata` được xử lý theo cơ chế run-once, không chạy liên tục sau khi seed dữ liệu.

7.6 Changed-Module Detection

Script `ci/detect-changed-modules.sh` giúp pipeline không rebuild toàn bộ monorepo khi chỉ đổi một module. Logic chính:

- Nếu chỉ đổi docs/spec/GitOps metadata thì trả về `__skip_full_ci__`.
- Nếu đổi Jenkinsfile, root `pom.xml`, `services.yaml`, Docker/K8s scripts thì chạy full path.
- Nếu đổi một module, chọn module đó và các service phụ thuộc dựa trên `services.yaml`.

Phần này đáng đưa vào báo cáo vì nó giải thích vì sao CD có thể scale trên monorepo YAS lớn mà không tốn thời gian build mọi service trong mỗi commit.

7.7 Build Và Runtime Fixes

App repo cũng có các thay đổi hỗ trợ CD chạy ổn định:

- Dockerfile của backend được chỉnh để copy đúng Spring Boot jar.
- `payment-paypal` được đóng gói đúng executable jar/thin jar theo nhu cầu runtime.
- Search integration tests được cô lập để tránh xung đột Testcontainers.
- Payment, product, tax, location, rating, webhook bổ sung test để giữ coverage gate.

Những thay đổi này không phải phần GitOps, nhưng là điều kiện để image sinh từ app repo có thể boot được trên K3s.

7.8 Observability Instrumentation Trong App Repo

App repo đã bổ sung nền observability ở tầng ứng dụng:

- Root `pom.xml` thêm `spring-boot-starter-actuator`, `micrometer-registry-prometheus` và `spring-boot-starter-opentelemetry`.
- Nhiều service expose `/actuator/prometheus` qua `management.endpoints.web.exposure.include=prometheus`.
- `SecurityConfig` cho phép Prometheus scrape `/actuator/prometheus` và health probes mà không cần login.
- App repo có `docker-compose.o11y.yml`, Prometheus config, Grafana datasource/dashboard, Loki, Tempo và OpenTelemetry Collector cho local observability.

Vì vậy chương Observability của báo cáo phải phân biệt hai phần: app repo đã có instrumentation; CD/K8s cần runtime evidence Prometheus/Grafana để chứng minh scrape và dashboard hoạt động trên cluster.

8 CD Repo Và GitOps Desired State

CD repo `emanhthangngot/yas-cd` là source of truth cho cluster. Jenkins không deploy trực tiếp vào namespace `dev`, `staging`, `developer`; Jenkins chỉ commit thay đổi vào CD repo, sau đó ArgoCD reconcile.

8.1 Cấu Trúc Desired State

Bảng 12: Vai trò các thư mục GitOps chính

Đường dẫn	Vai trò
<code>base/</code>	Config dùng chung cho application workloads: route config, compatibility service, shared config.
<code>platform/base/</code>	PostgreSQL, Redis, Kafka, Elasticsearch, Keycloak, ingress và namespace platform.
<code>overlays/dev</code>	Môi trường dev, auto-sync, nhận image tag từ main build.
<code>overlays/staging</code>	Môi trường staging, manual sync, chỉ dùng immutable release tag.
<code>overlays/developer</code>	Developer preview, dormant mặc định, active khi chạy <code>developer_build</code> .
<code>argocd/apps</code>	ArgoCD Application manifests trở về CD repo <code>main</code> .
<code>operations/sampled-data-seed</code>	Job seed dữ liệu một lần cho dev/staging.

8.2 CQ Service Scope

Theo file PDF phân loại service CQ, các service cốt lõi gồm `product`, `cart`, `order`, `customer`, `inventory`, `tax`, `media`, `search`, `storefront/backoffice BFF` và `UI`, `swagger UI`, `sampledata`. CD repo giữ thêm `location`, `payment`, `payment-paypal` vì chúng là runtime dependency của `tax/order flow`.

Bảng 13: Service scope runtime theo CQ demo

Nhóm service	Danh sách và lý do
Teacher-required	product, cart, order, customer, inventory, tax, media, search, storefront/backoffice BFF và UI, swagger-ui, sampledata.
Runtime dependency	location cho tax flow; payment và payment-paypal cho order/payment flow.
Disabled optional	promotion, rating, recommendation, webhook; giữ trong source/chart nhưng không bật mặc định để giảm tải VM đơn.
Run-once	sampledata không chạy liên tục; seed bằng operation riêng rồi đưa deployment về 0 replicas.

Các service optional như `promotion`, `rating`, `recommendation` và `webhook` không chạy trong base-line để giảm tải single-node, nhưng vẫn còn trong source và chart để mở rộng về sau.

8.3 Validation Gates Trong CD Repo

Trước khi commit GitOps, nhóm dùng các script:

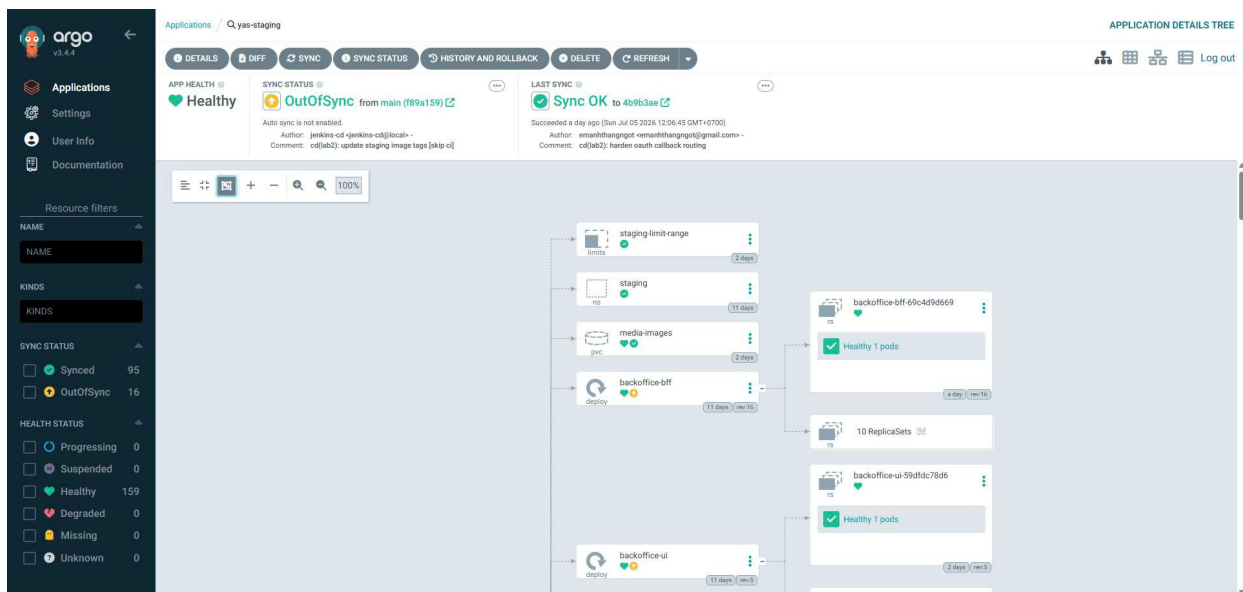
- `scripts/validate-gitops.sh`: render overlays, kiểm tra service catalog, secret-like patterns, sampledata operation overlays và active environment policy.
- `scripts/validate-staging-immutable.sh`: đảm bảo staging chỉ dùng release tag immutable, không dùng latest.
- `scripts/update-image-tag.sh`: contract cho Jenkins main/dev update.
- `scripts/promote-staging-release.sh`: đổi toàn bộ staging image tag sang vX.Y.Z.

```
1 == validate-gitops.sh @ 2026-07-05 commit 38b921d ==
2 validating overlay: dev
3 validating overlay: staging
4 validating overlay: developer
5 validating operation overlay: operations/sampledata-seed/dev
6 validating operation overlay: operations/sampledata-seed/staging
7 staging immutability check passed: all staging image tags are release tags
8 GitOps validation passed
```

Listing 2: GitOps validation passed trước khi commit desired state

8.4 ArgoCD Sync Policy

- **yas-dev**: automated sync/self-heal cho main branch changes.
- **yas-staging**: manual sync để operator approve release.
- **yas-developer**: automated sync nhưng dormant mặc định.
- **yas-platform**: quản lý shared infra.

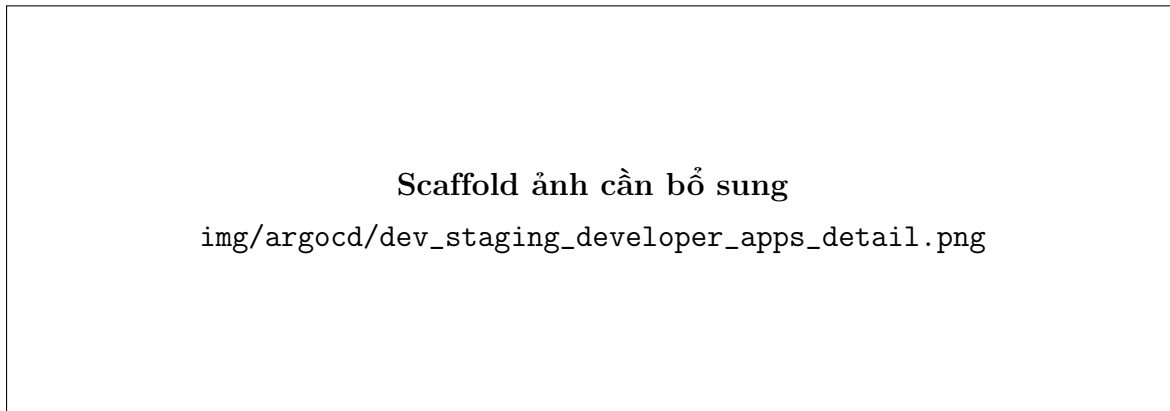


Hình 5: Staging release tạo diff trong ArgoCD trước khi operator sync thủ công

8.5 Active Environment Policy

Vì GCP VM chỉ là single-node lab cluster, CD repo không cố chạy toàn bộ mọi profile cùng lúc. Policy hiện tại được encode trong overlays và runbook:

- Baseline: **dev** active, **staging** active, **developer** dormant.
- Khi chạy developer preview: **developer** active, **staging** dormant để tránh nhân đôi Java pods.
- Sau teardown: **developer** quay về dormant, **dev** + **staging** trở lại baseline.
- Sampledata luôn là seed operation có kiểm soát, không phải service chạy thường trực.



Hình 6: Scaffold ảnh detail ArgoCD cho yas-dev, yas-staging, yas-developer

9 Các Luồng Jenkins CD

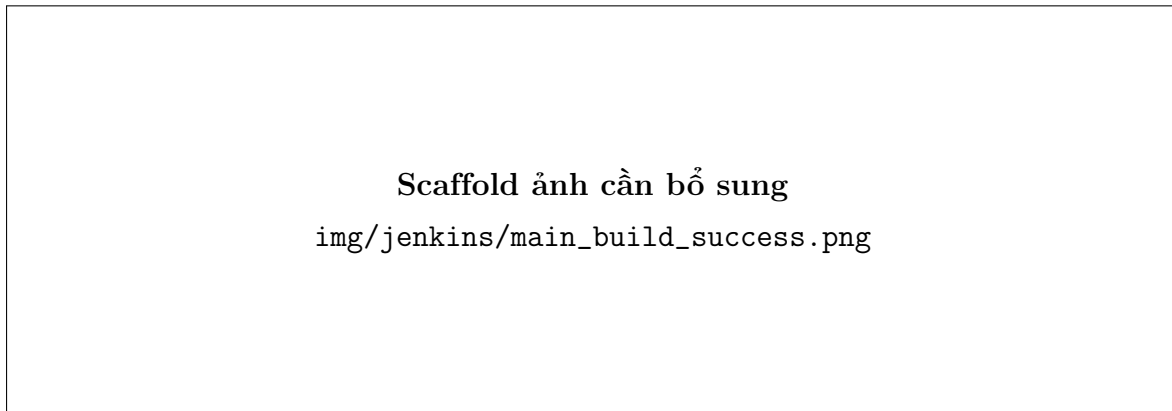
Đồ án có ba luồng CD chính: dev auto deploy, staging release có duyệt, và developer preview on-demand. Điểm chung là tất cả đều đi qua GitOps commit; không có job nào trực tiếp apply manifest vào namespace do ArgoCD quản lý.

9.1 Dev Auto Deploy

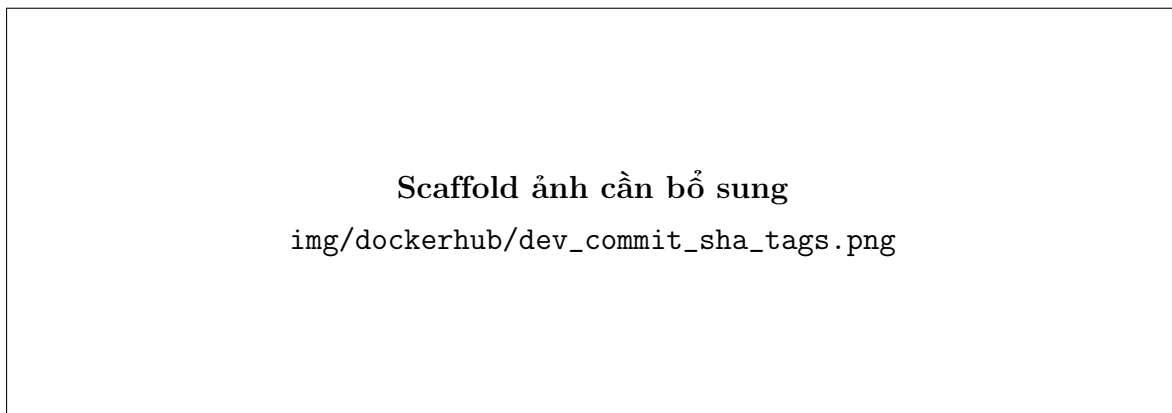
Khi app repo main thay đổi, Jenkins:

1. Detect modules thay đổi và chạy CI gates.
2. Build/push Docker image cho service deployable với tag short commit SHA.
3. Push thêm tag tiện dụng `main` và `latest`.
4. Clone CD repo, chạy `scripts/update-image-tag.sh dev <service> <sha>`.
5. Commit `cd(lab2): update dev image tags [skip ci]` vào `yas-cd/main`.
6. ArgoCD `yas-dev` tự sync.

Việc overlay dev dùng commit SHA thay vì `latest` giúp mỗi deploy có dấu vết rõ ràng và tránh tình trạng Kubernetes không rollout vì manifest không đổi.



Hình 7: Scaffold ảnh Jenkins main build chạy qua CI gates và Docker push



Hình 8: Scaffold ảnh Docker Hub hiển thị image tag theo commit SHA cho dev

9.2 Staging Release

Staging chỉ nhận release tag vX.Y.Z. Jenkins release tag build là promote-only:

1. Tag vX.Y.Z trigger Jenkins multibranch/tag job.
2. Pipeline bỏ qua test/scan/build lại source vì source đã được gate-check bởi main build.
3. Jenkins đọc tag nguồn từ service từ overlays/dev.
4. Dùng `docker buildx imagetools create retag` image hiện có sang vX.Y.Z.
5. Chạy `scripts/promote-staging-release.sh vX.Y.Z`.
6. Commit staging overlay vào CD repo.

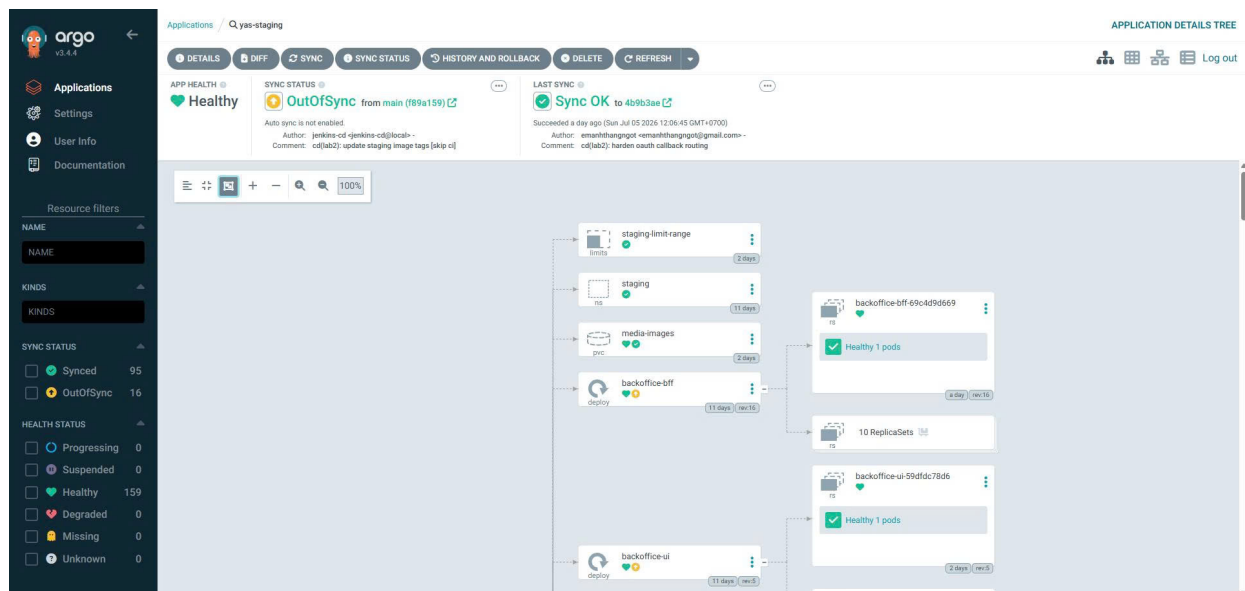
7. Operator review diff và sync yas-staging thử công.

```
1 == Demo prep: align yas-staging Application with Git (manual-sync) | 2026-07-05 ==
2 Truoc: {"automated":{"prune":true,"selfHeal":true},"syncOptions":["CreateNamespace=true"]}
3 Lenh: kubectl apply Application yas-staging theo ban origin/main (bo khoi automated)
4 Sau: {"syncOptions":["CreateNamespace=true"]}
5 Y nghĩa: release staging can nguoi duyet sync (dung runbook staging-developer-runbook.md)
6 Ghi chu: day cung la vi du drift giua Git va cluster o tang Application manifest -
7 cac file argocd/apps/*.yaml phai kubectl apply tay, khong tu dong sync.
```

Listing 3: Operator sync staging sau khi release GitOps diff được tạo

Scaffold ảnh cần bổ sung
img/jenkins/release_v012_success.png

Hình 9: Scaffold ảnh Jenkins release tag v0.1.2 promote thành công



Hình 10: ArgoCD staging có diff trước khi operator sync thử công

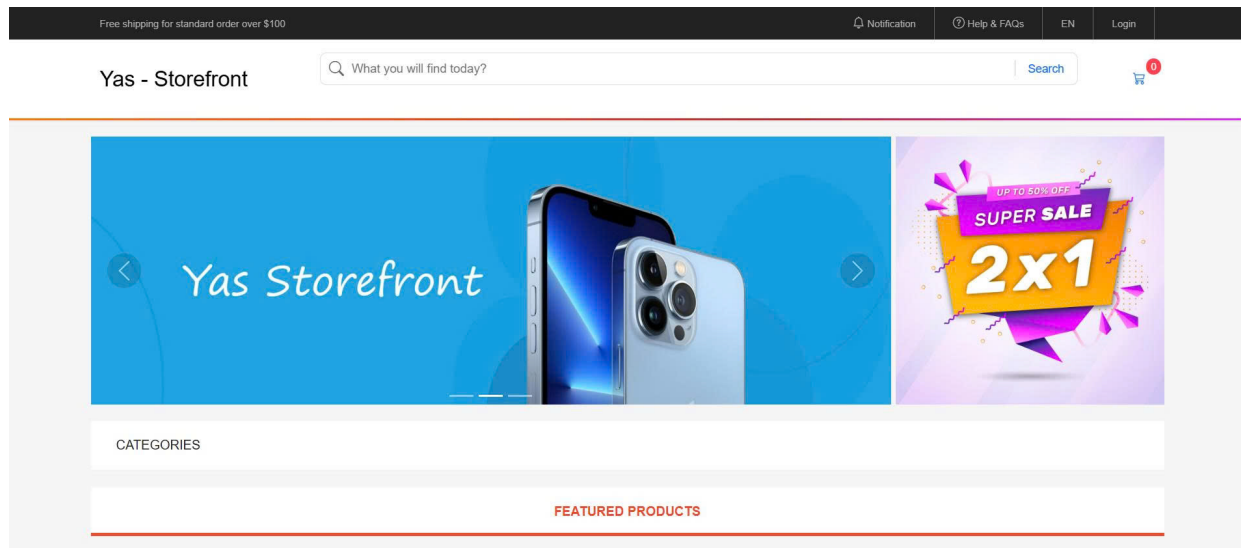
Bảng 14: Quá trình hardening staging release

Lần chạy	Kết quả	Bài học kỹ thuật
v0.1.0	GitOps gate chặn.	Tag đóng vào commit cũ làm lệch service scope giữa app repo và CD repo.
v0.1.1 lần 1	Jenkins fail do thiếu buildx.	Release promote cần agent có Docker buildx/imagetools.
v0.1.1 lần 2	Fail do mixed dev tags.	Không thể giả định mọi service trong dev cùng một SHA.
v0.1.2	Thành công.	Promote theo tag nguồn từng service đọc từ dev overlay; release tag là promote-only.

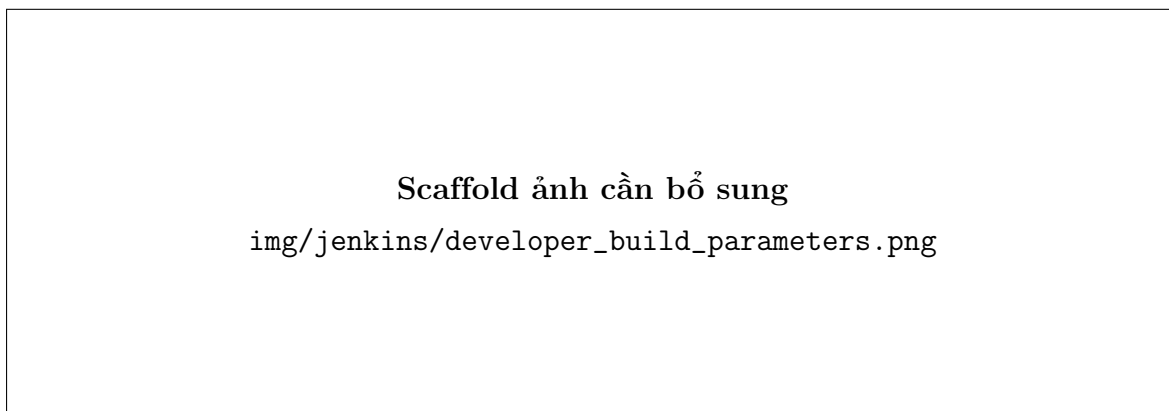
9.3 Developer Preview

Yêu cầu đề bài có job `developer_build` để developer nhập branch cho service muốn test. Nhóm tách job này sang CD repo để giữ main app pipeline đơn giản:

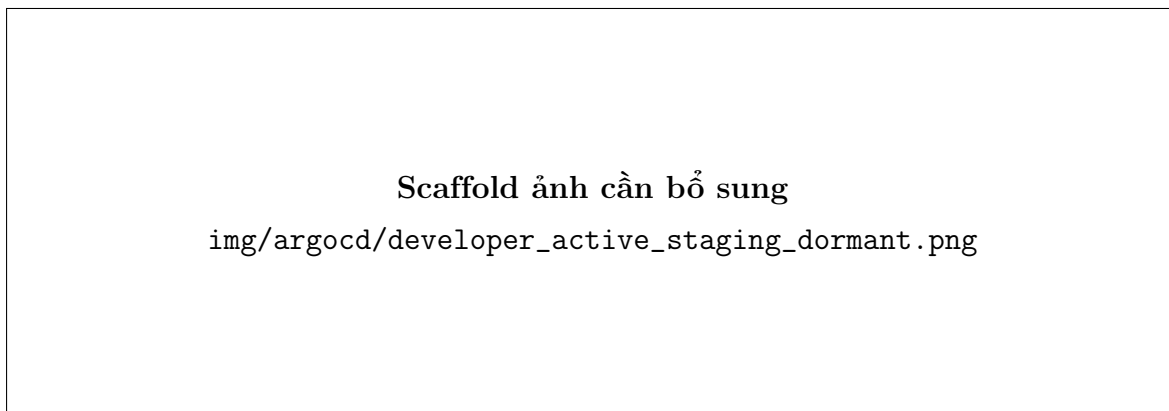
- App repo feature branch chỉ build/push image commit SHA.
- CD repo `Jenkinsfile.developer-build` resolve branch sang SHA và kiểm tra image đã tồn tại trên Docker Hub.
- Job gọi `scripts/prepare-developer-preview.sh tax=<sha>` hoặc nhiều assignment khác.
- `developer` active, `dev` vẫn active, `staging` dormant để tránh quá tải.
- ArgoCD tự sync `yas-developer`; developer truy cập <http://storefront.developer.yas.local.com:30846/>.



Hình 11: Developer preview truy cập được qua NodePort 30846



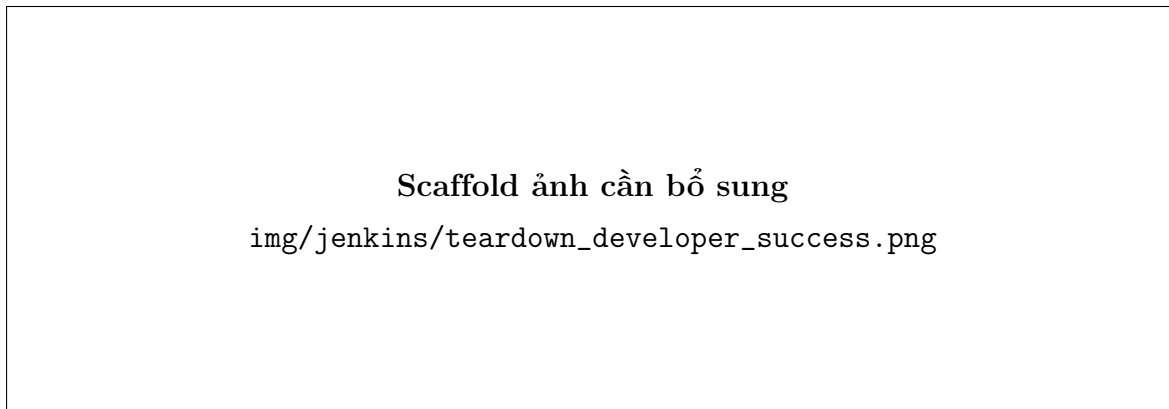
Hình 12: Scaffold ảnh job developer_build với parameter branch/service



Hình 13: Scaffold ảnh ArgoCD/Kubernetes khi developer active và staging dormant

9.4 Teardown Developer

Job `teardown_developer` chạy từ CD repo, yêu cầu confirm `developer-teardown`. Job reset overlay `developer` về tag `main`, đưa `developer` về dormant và khôi phục baseline `dev + staging active`. Đây là cách đáp ứng yêu cầu xóa phần triển khai `developer` mà vẫn không dùng `kubectl delete` trực tiếp.



Hình 14: Scaffold ảnh job teardown trả developer về dormant

9.5 Bằng Chứng Demo 3 Luồng

Evidence tổng hợp cho ba luồng nằm trong `evidence/35_cd_demo_summary.md`: `dev auto deploy`, `staging release v0.1.2`, và `developer_build/teardown`. Các log lỗi trước đó cũng được giữ lại để giải thích quá trình hardening release flow: `v0.1.0` bị gate chặn, `v0.1.1` lỗi buildx/mixed tag, `v0.1.2` thành công sau khi sửa promote source.

10 Runtime Verification Và Service Mesh

10.1 Platform Readiness

Platform stack gồm PostgreSQL, Redis, Kafka, Elasticsearch và Keycloak. Các service YAS ở `dev` và `staging` dùng chung platform nhưng tách database theo environment. Evidence hiện có xác nhận node, storage class, PVC và database đã sẵn sàng.

```
xuantri@gcp-ci-cd-agent:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1460 qdisc mq state UP group default qlen 1000
    link/ether 42:01:0a:80:00:02 brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    inet 10.128.0.2/32 metric 100 scope global dynamic ens4
        valid_lft 2519sec preferred_lft 2519sec
    inet6 fe80::4001:aff:fe80:2/64 scope link
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether fe:97:4b:f7:d1:98 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
4: flannel.1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1410 qdisc noqueue state UNKNOWN group default
    link/ether 06:b3:3f:a0:fc:f7 brd ff:ff:ff:ff:ff:ff
    inet 10.42.0.0/32 scope global flannel.1
        valid_lft forever preferred_lft forever
    inet6 fe80::4b3:3fff:fea0:fcf7/64 scope link
        valid_lft forever preferred_lft forever
5: cni0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1410 qdisc noqueue state UP group default qlen 1000
    link/ether e2:fe:cf:d5:6e:a5 brd ff:ff:ff:ff:ff:ff
    inet 10.42.0.1/24 brd 10.42.0.255 scope global cni0
        valid_lft forever preferred_lft forever
    inet6 fe80::e0fe:cfff:fed5:6ea5/64 scope link
        valid_lft forever preferred_lft forever
8: veth30c69ee20i1f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1410 qdisc noqueue master cni0 state UP group default
    link/ether 9e:6d:b2:54:b2:44 brd ff:ff:ff:ff:ff:ff link-netns cni-ef2e68b5-4251-4e77-3042-7ec6744b2559
    inet6 fe80::9c6d:b2ff:fe54:b244/64 scope link
        valid_lft forever preferred_lft forever
9: veth2a71f7680i1f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1410 qdisc noqueue master cni0 state UP group default
```

Hình 15: External IP của GCP VM dùng cho NodePort demo

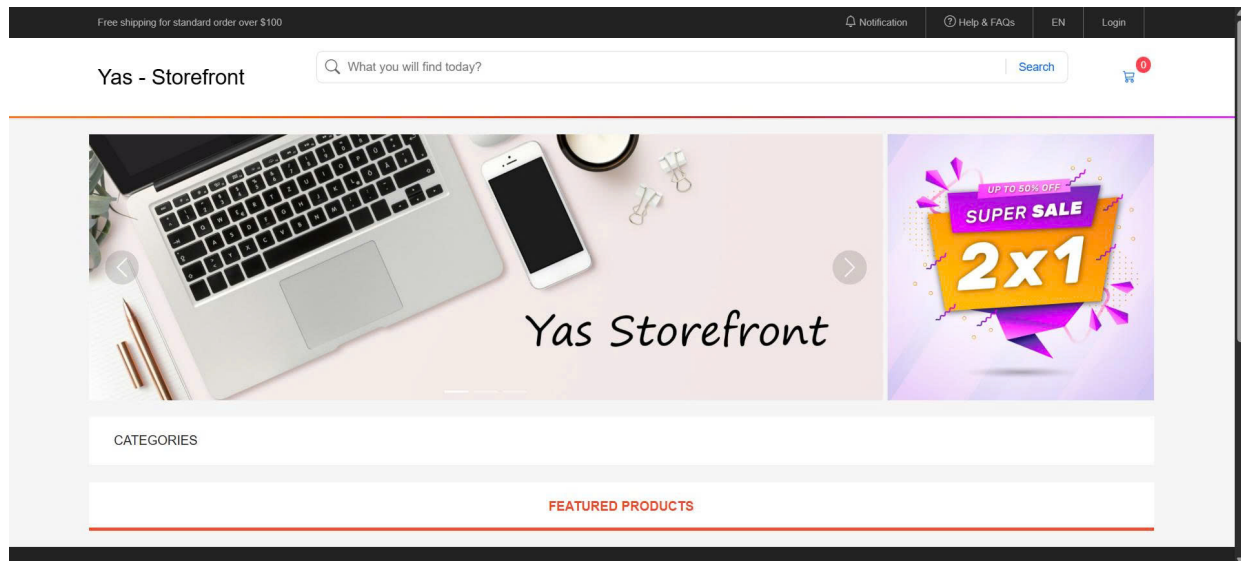
10.2 External Access

Runtime access chính dùng Nginx ingress NodePort 30846 thông qua cơ chế phân phối Multi-Host Ingress, định tuyến bằng trường Host header. Các tên miền dùng chung cũ (yas.<env>.local) đã được loại bỏ hoàn toàn để phân tách luồng nghiệp vụ giữa storefront, backoffice, swagger và identity. Developer hoặc giảng viên thêm host mapping vào tệp hosts:

```
1 34.124.212.254 storefront.dev.yas.local.com backoffice.dev.yas.local.com swagger
    .dev.yas.local.com identity.dev.yas.local.com
2 34.124.212.254 storefront.staging.yas.local.com backoffice.staging.yas.local.com
    swagger.staging.yas.local.com identity.staging.yas.local.com
3 34.124.212.254 storefront.developer.yas.local.com backoffice.developer.yas.local
    .com swagger.developer.yas.local.com identity.developer.yas.local.com
```

Listing 4: Cấu hình tệp hosts phân giải tên miền mới

Storefront dev/staging đã được smoke test qua curl theo tên miền storefront.<env>.yas.local.com: HTML trả 200, product API trả seeded iPhone data, media thumbnail trả image/jpeg, và Keycloak login/register render đúng host cùng NodePort.



Hình 16: Storefront dev reachable qua NodePort

10.3 Service Mesh Scope

Mesh được áp dụng cho dev và staging, không chỉ một namespace demo tách biệt. Các pod ứng dụng có workload container và istio-proxy, do đó trạng thái READY kỳ vọng là 2/2. Mesh policy gồm:

- PeerAuthentication cho STRICT mTLS ở backend path.
- DestinationRule cho traffic mesh.
- VirtualService cho retry/fault evidence.
- AuthorizationPolicy theo service identity để có allow/deny rõ ràng.

Bảng 15: Service mesh requirement và implementation

Yêu cầu mesh	Triển khai trong CD repo	Evidence cần trình bày
mTLS	overlays/dev/istio/mtls.yaml và overlays/staging/istio/mtls.yaml dùng PeerAuthentication STRICT và DestinationRule ISTIO_MUTUAL.	Screenshot pod READY 2/2, manifest mTLS, Kiali security lock nếu có.
Retry	virtual-services.yaml khai báo retry cho critical services với retryOn: 5xx,connect-failure,refused-stream.	Manifest VirtualService và log/curl test retry.
Authorization	authorization-policies.yaml giới hạn service-to-service bằng service account/SPIFFE principal.	Curl allow/deny: tax -> location allowed, cart -> location denied 403.
Topology	Kiali quan sát graph traffic dev/staging.	Screenshot Kiali topology hiển thị store-front/BFF/backend path.

Scaffold ảnh cần bổ sung
img/mesh/mtls_strict_pods_2of2.png

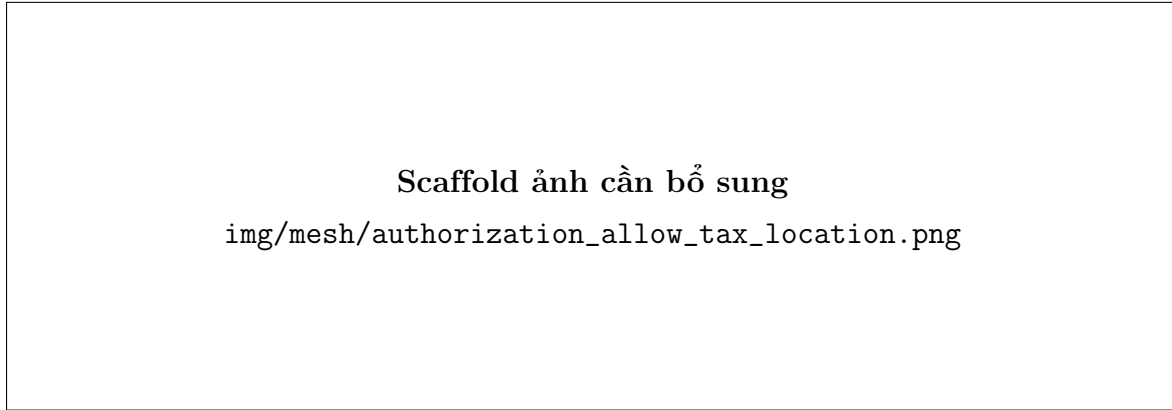
Hình 17: Scaffold ảnh pod có workload container và Istio sidecar READY 2/2

10.4 AuthorizationPolicy Evidence

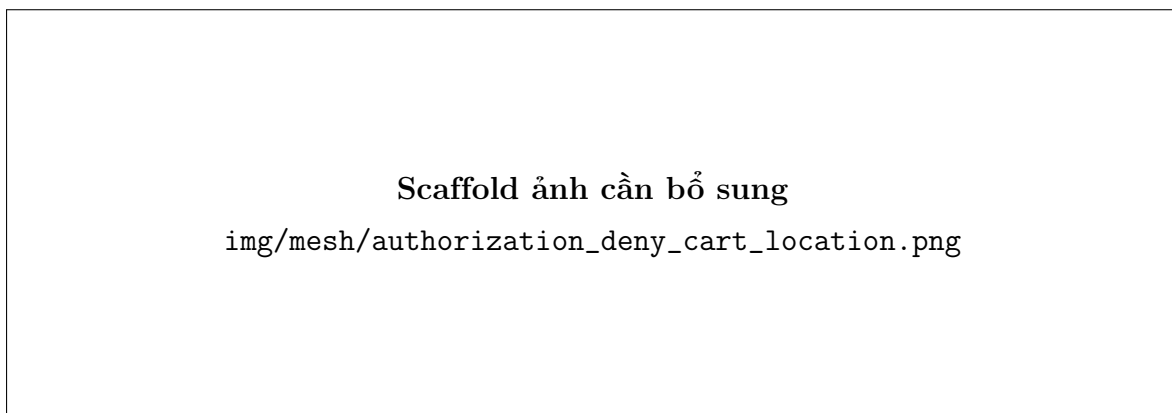
Kịch bản allow/deny dùng đường gọi tax -> location:

- Gọi từ tax sang location: request đi qua mesh và đến application layer.
- Gọi từ cart sang location: bị chặn HTTP 403 bởi Istio AuthorizationPolicy.

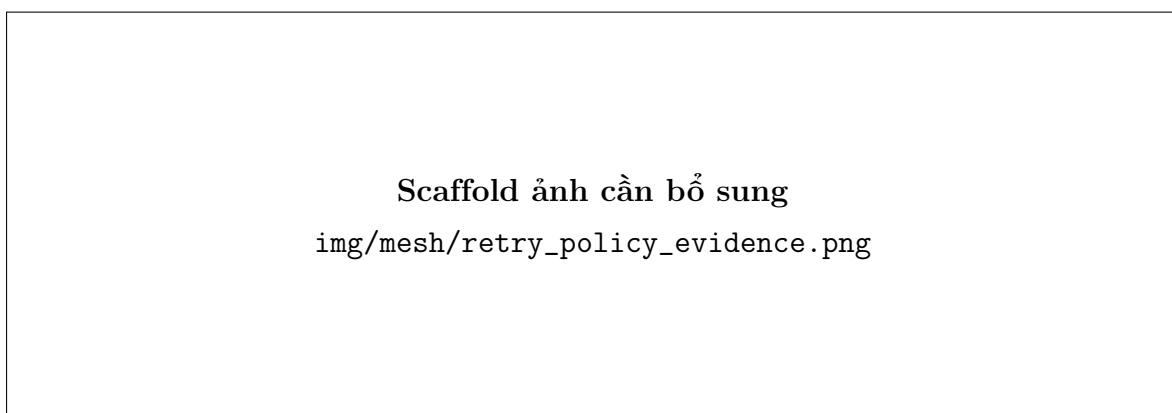
HTTP 403 trong case denied là bằng chứng policy hoạt động ở mesh layer. HTTP 500 ở case allowed vẫn có giá trị vì nó chứng minh request đã qua được mesh và chạm vào Spring Boot application, lỗi còn lại thuộc application endpoint.



Hình 18: Scaffold ảnh curl allow: tax gọi location



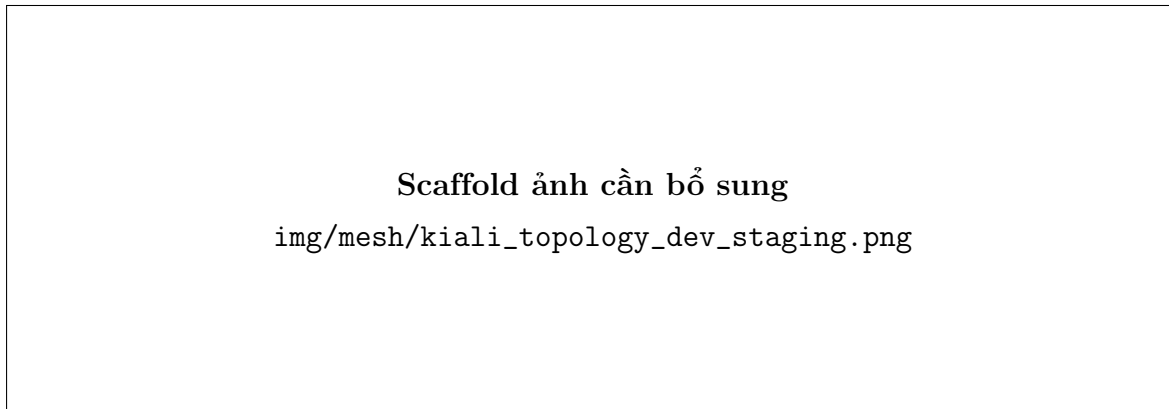
Hình 19: Scaffold ảnh curl deny: cart gọi location bị Envoy RBAC 403



Hình 20: Scaffold ảnh retry policy hoặc log retry cho service trong critical path

10.5 Kiali Topology

Kiali được dùng để quan sát topology service mesh. Nếu chưa có ảnh cuối cùng trong `latex-report/img/mesh/`, báo cáo cần để placeholder rõ: chụp Kiali topology cho `dev` hoặc `staging`, hiển thị luồng storefront/BFF/backend và sidecar traffic.



Hình 21: Placeholder ảnh Kiali topology cần bổ sung trước khi nộp

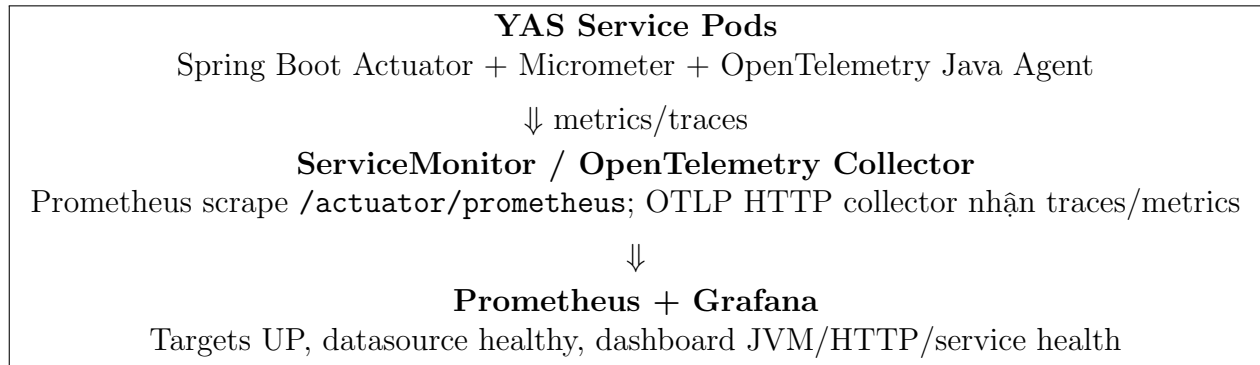
11 Observability Với Prometheus Và Grafana

Đề bài gốc ghi không bắt buộc triển khai Grafana/Prometheus cho phần cơ bản, nhưng nhóm bổ sung phần observability để hoàn thiện khả năng vận hành. Nội dung này gồm hai lớp: instrumentation trong app repo và scrape/dashboard trong runtime.

11.1 Mục Tiêu Observability

Observability trong đồ án không chỉ để có dashboard đẹp, mà để trả lời ba câu hỏi vận hành:

- Service còn sống không: health/readiness/liveness và pod status.
- Service có phục vụ request không: HTTP request rate, latency, error rate.
- JVM/Spring Boot có quá tải không: heap, thread, class count, GC và actuator metrics.



Hình 22: Luồng observability từ service đến dashboard

11.2 Instrumentation Trong App Repo

App repo `tzin1401/yas` đã được bổ sung các dependency dùng chung ở root `pom.xml`:

- `spring-boot-starter-actuator`: mở `health/metrics` endpoint.
- `micrometer-registry-prometheus`: xuất metric theo format Prometheus.
- `spring-boot-starter-opentelemetry`: export trace qua OTLP.

Các service Spring Boot expose `/actuator/prometheus` và `/actuator/health/**`. `SecurityConfig` cho phép hai endpoint này để Prometheus scrape không bị chặn bởi authentication. Đây là thay đổi quan trọng ở app repo và phải được trình bày trong báo cáo, vì Prometheus/Grafana không thể có dữ liệu nếu ứng dụng không xuất metric.

11.3 Tự Động Tiêm Agent OpenTelemetry (Auto-Instrumentation)

Để tối ưu hóa luồng đo lường trên môi trường `dev` mà không cần build lại các Docker image của ứng dụng, nhóm đã triển khai giải pháp tự động tiêm OpenTelemetry Java Agent thông qua `OpenTelemetry Operator`:

- Cơ chế hoạt động: `OpenTelemetry Operator` theo dõi tài nguyên của namespace. Khi phát hiện các Deployment được chỉ định trong tệp `otel-inject-patch.yaml` có khai báo annotation `instrumentation.opentelemetry.io/inject-java:"yas"`, Operator sẽ tự động chèn một `initContainer` để sao chép tệp JAR của OTEL agent vào thư mục chạy, đồng thời tự động cấu hình các biến môi trường `JAVA_TOOL_OPTIONS` và `OTEL_*` tương ứng trên Pod.

- Khai báo Instrumentation CR: Nhóm định nghĩa tài nguyên `Instrumentation` trong `observability/instrumentation.yaml` trở về endpoint của OpenTelemetry Collector chạy tại địa chỉ `http://opentelemetry-collector.observability:4318` sử dụng giao thức `http/protobuf`.
- Khớp chuẩn SemConv: Tác nhân OpenTelemetry được cấu hình xuất dữ liệu đo lường định dạng OTLP trực tiếp, tự động sinh các metric JVM tiêu chuẩn (như `jvm_class_count`, `jvm_thread_count`, `http_server_request_duration_seconds`) trùng khớp hoàn toàn với bảng điều khiển mẫu `observability\_dashboard.json` của ứng dụng YAS.

11.4 Scrape Contract Trong CD Repo

Backend Helm chart trong CD repo có template `ServiceMonitor` khi `serviceMonitor.enabled=true`. Contract scrape:

- Kind: `ServiceMonitor`.
- Label: `release=prometheus`.
- Endpoint path: `/actuator/prometheus`.
- Port: `metric`, tương ứng management port của service.

CD repo cũng cấu hình application config để expose `prometheus`, `health` và đặt management server port riêng. Như vậy app repo cung cấp metric, CD repo cung cấp cách Prometheus operator phát hiện và scrape service.

11.5 Local Observability Assets Từ App Repo

App repo có sẵn local LGTM stack:

- `docker-compose.o11y.yaml`: OpenTelemetry Collector, Prometheus, Grafana, Loki, Tempo.
- `docker/prometheus/prometheus.yaml`: scrape Prometheus/self và collector.
- `docker/grafana/provisioning/datasources`: datasource Prometheus/Loki/Tempo.

- `docker/grafana/provisioning/dashboards`: dashboard observability và Prometheus.

Phần này là nguồn thiết kế tốt để giải thích dashboard và PromQL trong báo cáo, nhưng runtime K3s vẫn cần evidence riêng nếu nhóm muốn claim đã triển khai Prometheus/Grafana trên cluster.

11.6 Runtime Evidence Cho Observability

Để claim observability đã chạy trên K3s, báo cáo cần đủ bốn lớp evidence:

Bảng 16: Checklist runtime observability evidence

Lớp	Cần chứng minh	Ảnh/log nên có
Kubernetes	Pod Prometheus/Grafana/collector chạy ổn định.	<code>kubectl get pods -n observability, Grafana/Prometheus service.</code>
Prometheus scrape	Target/service monitor cho YAS ở trạng thái UP.	<code>img/observability/prometheus_targets_yas.png.</code>
Grafana data-source	Datasource Prometheus healthy.	<code>img/observability/grafana_datasource_healthy.png.</code>
Dashboard	Dashboard có JVM/HTTP/service metrics của ít nhất một YAS backend.	<code>img/observability/grafana_yas_metrics_dashboard.png.</code>

11.7 Evidence Cần Bổ Sung

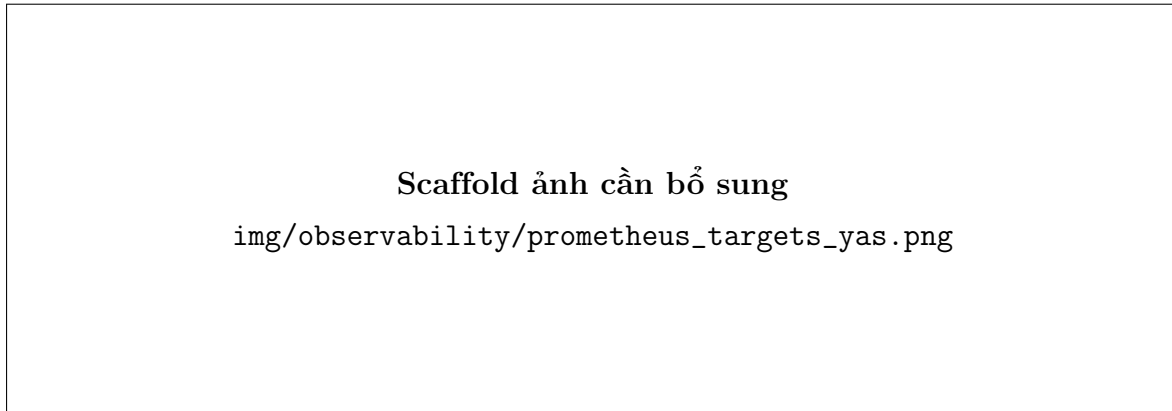
Hiện tại báo cáo có thể ghi chắc các phần sau:

- App repo đã có Actuator, Prometheus registry và OpenTelemetry dependency.
- Service config/security cho phép endpoint `/actuator/prometheus`.
- CD chart có ServiceMonitor contract.

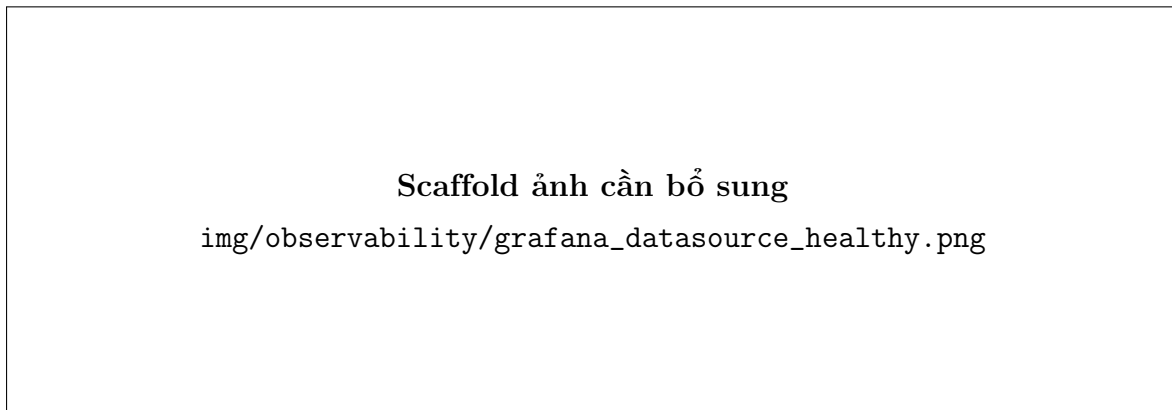
Các evidence cần chụp trước khi nộp nếu muốn đánh dấu runtime verified:

- Prometheus UI: Targets hoặc ServiceMonitor target ở trạng thái UP.
- Grafana UI: datasource Prometheus healthy.

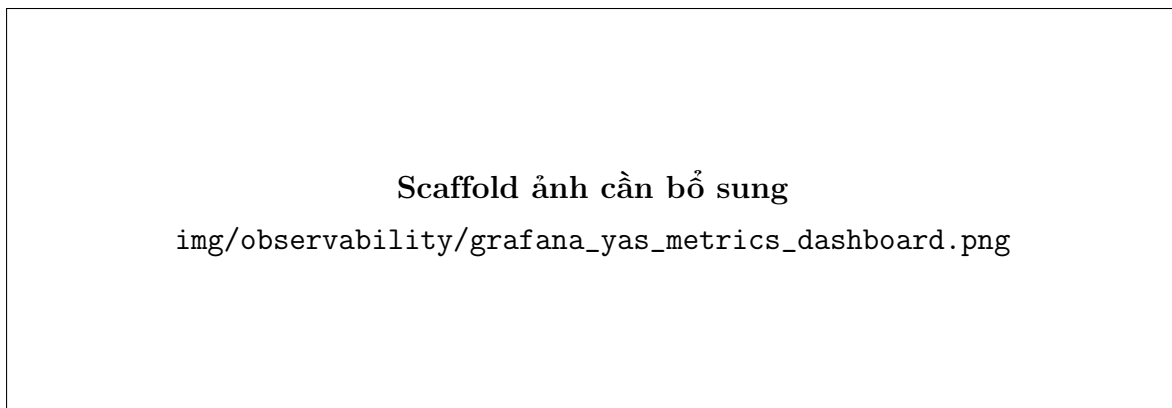
- Grafana dashboard: JVM memory, HTTP request rate, service status hoặc pod/service metrics.
- Optional: Tempo trace hoặc Kiali/Grafana correlation nếu đã bật OTLP collector trên K3s.



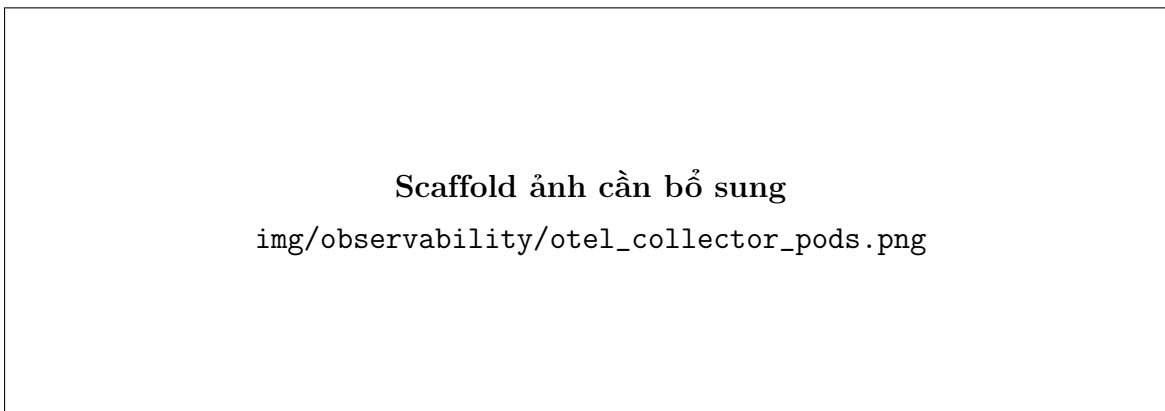
Hình 23: Placeholder ảnh Prometheus targets cần bổ sung



Hình 24: Scaffold ảnh Grafana datasource Prometheus healthy



Hình 25: Placeholder ảnh Grafana dashboard cần bổ sung



Hình 26: Scaffold ảnh OpenTelemetry Collector/Operator chạy trên cluster

12 Evidence Tổng Hợp Và Known Issues

12.1 Evidence Đã Có

Bảng 17: Nhóm evidence hiện có	
Nhóm	Evidence tiêu biểu
Hạ tầng	01_gcp_vm_specs.png, 02_gcp_vm_ip_addr.png, tool gate và PVC/database output trong .agents/evidence/README.md.
ArgoCD/GitOps	06_argocd_apps_overview.png, 07_validate_gitops_passed.txt, 11_gitops_structure_review.txt.
Staging release	21_jenkins_v010_gate_blocked.txt, 21b_jenkins_v011_buildx_missing.txt, 21c_jenkins_v011_mixed_tag_promote_fail.txt, 21d_jenkins_v012_release_success.txt.
Developer preview	28_developer_build_console.txt, 29_preview_commit.txt, 31_developer_up_staging_down.txt, 32_developer_reachable.png, 33_teardown_console.txt.
Security	12_gitleaks_yas_cd.json, 13_gitleaks_yas_app_repo.redacted.json, 14_gitleaks_triage_summary.md, 15_port_exposure_check.txt.
Runtime smoke	17_app_reachable_dev.png, 35_cd_demo_summary.md, app/API/Keycloak smoke output trong .agents/evidence/README.md.

12.2 Ảnh Evidence Đã Đưa Vào Report

Các ảnh dưới đây đã được copy vào `latex-report/img` để PDF không chỉ trỏ tới file evidence bên ngoài:

Bảng 18: Ảnh evidence thật đang hiển thị trong PDF

Ảnh trong report	Nguồn gốc	Ý nghĩa
<code>img/platform/01_gcp_vm_specs.png</code>	<code>evidence/01_gcp_vm_specs.png</code>	VM 32GB RAM dùng làm Jenkins agent và K3s node.
<code>img/platform/02_gcp_vm_ip_addr.png</code>	<code>evidence/02_gcp_vm_ip_addr.png</code>	External IP dùng cho NodePort demo.
<code>img/argocd/06_argocd_apps_overview.png</code>	<code>evidence/06_argocd_apps_overview.png</code>	ArgoCD apps overview.
<code>img/runtime/17_app_reachable_dev.png</code>	<code>evidence/17_app_reachable_dev.png</code>	Storefront dev reachable.
<code>img/argocd/24_argocd_outofsync_diff.png</code>	<code>evidence/24_argocd_outofsync_diff.png</code>	Staging release diff trước manual sync.
<code>img/developer/32_developer_reachable.png</code>	<code>evidence/32_developer_reachable.png</code>	Developer preview reachable.

12.3 Ảnh Cần Chụp Bổ Sung Theo Ưu Tiên

Bảng 19: Backlog ảnh cần bổ sung trước khi nộp

Ưu tiên	Ảnh cần chụp	Lý do
P0	Jenkins main build success và release v0.1.2.	Chứng minh pipeline chạy thật thay vì chỉ có log text.
P0	ArgoCD detail cho yas-dev, yas-staging, yas-developer.	Chứng minh sync policy và health từng môi trường.
P0	Docker Hub tags cho commit SHA và release tag.	Chứng minh branch/main/release image tagging đúng đề bài.
P1	Kiali topology, mTLS 2/2, allow/deny curl.	Củng cố phần nâng cao service mesh.
P1	Prometheus targets và Grafana dashboard.	Chỉ khi có hai ảnh này mới nên claim observability runtime verified.
P2	Teardown developer console và baseline sau teardown.	Làm rõ yêu cầu xóa preview trong đề bài.

12.4 Các Sự Cố Kỹ Thuật Và Giải Pháp Khắc Phục (Troubleshooting)

Trong quá trình triển khai hệ thống CD và GitOps cho ứng dụng YAS trên hạ tầng single-node K3s, nhóm đã gặp một số sự cố kỹ thuật và đã nghiên cứu, áp dụng các giải pháp khắc phục triệt để:

- Lỗi Kustomize thiếu Helm và Load Restrictor trong ArgoCD: Khi ArgoCD dịch kustomization.yaml trong thư mục base/, hệ thống báo lỗi không thể sử dụng HelmChartInflation-Generator do thiếu cờ `--enable-helm`, đồng thời chặn việc tham chiếu đến thư mục chart nằm ngoài thư mục base (lỗi bảo mật Load Restrictor). Nhóm đã giải quyết bằng cách cấu hình lại ConfigMap `argocd-cm` trong namespace `argocd`:

```
1 kubectl patch configmap argocd-cm -n argocd -p '{"data": {"kustomize.  
    buildOptions": "--enable-helm --load-restrictor=LoadRestrictionsNone"}}'  
2 kubectl rollout restart deployment argocd-repo-server -n argocd  
3
```

- Lỗi kẹt ứng dụng ArgoCD ở trạng thái Progressing do IP Ingress: Trên môi trường K3s single-node không có cloud load balancer, dịch vụ Ingress controller luôn ở trạng thái `<pending>`

để nhận External IP. Điều này khiến ArgoCD coi Ingress ở trạng thái **Progressing** vô thời hạn. Nhóm đã patch tham số của controller từ **--publish-service** sang **-report-node-internal-ip-address** để controller báo cáo trực tiếp IP của node:

```
1 kubectl patch deploy ingress-nginx-controller -n ingress-nginx --type=json
  -p ' [{"op": "replace", "path": "/spec/template/spec/containers/0/args/1", "
    value": "--report-node-internal-ip-address"} ] '
```

Giải pháp này giúp các Ingress nhận được địa chỉ IP node và các ứng dụng ArgoCD chuyển sang trạng thái **Healthy** thành công.

3. Lỗi Envoy RBAC 403 Forbidden do sai cấu hình Gateway Route: Sau khi bật Authorization-Policy để kiểm soát kết nối, mọi request đến API qua NodePort đều bị chặn HTTP 403. Truy vết Envoy logs cho thấy request đến từ ServiceAccount **default** của **nginx-api-gateway** thay vì **storefront-bff**. Nguyên nhân do Spring Cloud Gateway đổi cấu hình từ **spring.cloud.gateway.routes** sang cấu hình WebFlux **spring.cloud.gateway.server.webflux.routes**, khiến cấu hình route cũ bị bỏ qua âm thầm và traffic đi vòng qua api-gateway cũ. Nhóm đã sửa đổi script **sync-gateway-routes.sh** để đồng bộ định dạng cấu hình mới nhất, cập nhật assertion kiểm tra và khởi động lại BFF để nhận ConfigMap mới.
4. Lỗi quá tải kết nối cơ sở dữ liệu PostgreSQL: Với hạ tầng hạn chế của cụm lab single-node, việc khởi động đồng thời nhiều Spring Boot microservices thực hiện Liquibase migration khiến PostgreSQL hết connection khả dụng, trả về lỗi **too many clients already**. Nhóm đã giảm cấu hình kết nối trong ConfigMap chung: đặt **maximum-pool-size:2** và **minimum-idle:0** cho Hikari CP, đồng thời tăng tham số **max_connections** của container PostgreSQL lên 500 để đáp ứng tải khởi động.
5. Các dịch vụ khởi động chậm bị Liveness Probe khởi động lại liên tục: Khi nhiều dịch vụ chạy Liquibase/JPA đồng thời trên 1 node VM, CPU bị quá tải tạm thời khiến cổng actuator metric phản hồi chậm hơn thời gian thăm dò của liveness probe. Hậu quả là container bị Kubernetes tiêu diệt liên tục (Exit Code 137). Nhóm đã cấu hình thêm **startupProbe** cho biểu đồ Helm chart backend, cho phép tăng thời gian chờ khởi động lần đầu lên tối đa 5 phút trước khi liveness/readiness probe bắt đầu giám sát định kỳ.

12.5 Known Issues Và Cách Trình Bày Trung Thực

- Snyk tạm tắt trong Jenkinsfile hiện tại: báo cáo chỉ ghi Gitleaks, test, coverage và SonarQube đang nằm trong path chính; Snyk là gate kế thừa cần bật lại sau khi CD ổn định.
- Payment staging Liquibase checksum: release v0.1.2 đã promote/deploy đúng image, nhưng service payment có lỗi migration checksum khi restart trên DB bền. Đây là vấn đề runtime/application migration, không phải lỗi GitOps release flow.
- Plaintext lab secrets: CD repo còn placeholder Kubernetes Secret dạng lab value; cần ghi đây là lab-only và production nên chuyển sang SealedSecrets/External Secrets.
- Observability runtime: app instrumentation và ServiceMonitor contract đã có, nhưng nếu chưa chụp Prometheus/Grafana trên K3s thì không đánh dấu runtime verified.
- Kiali topology screenshot: mesh policy evidence đã có; ảnh topology cuối cùng cần bổ sung nếu chưa nằm trong report.

12.6 Kết Luận

Hệ thống hiện đã đáp ứng trọng tâm Đồ án 2: Jenkins sinh image từ app repo, Docker Hub lưu tag theo commit/release, CD repo quản lý desired state, ArgoCD sync cluster, dev/staging/developer có chính sách rõ ràng, và service mesh đã có evidence mTLS, retry, allow/deny. Phần observability đã có nền ở source và chart; bước còn lại là chụp evidence runtime Prometheus/Grafana để hoàn thiện báo cáo trước khi nộp.

Tài liệu

- [1] Istio Authors. Istio service mesh documentation. <https://istio.io/latest/docs/>, 2026. Accessed: 2026-07-07.
- [2] OpenTelemetry Authors. Opentelemetry operator for kubernetes. <https://opentelemetry.io/docs/kubernetes/operator/>, 2026. Accessed: 2026-07-07.
- [3] Rancher Labs. K3s - lightweight kubernetes. <https://docs.k3s.io/>, 2026. Accessed: 2026-07-07.
- [4] Argo Project. Argo cd documentation. <https://argo-cd.readthedocs.io/>, 2026. Accessed: 2026-07-07.
- [5] Jenkins Project. Jenkins user documentation. <https://www.jenkins.io/doc/>, 2026. Accessed: 2026-07-07.
- [6] OpenGitOps Project. Gitops principles v1.0.0. <https://opengitops.org/>, 2022. Accessed: 2026-07-07.
- [7] SonarSource. Sonarqube documentation. <https://docs.sonarqube.org/>, 2026. Accessed: 2026-07-07.